NEXUS

Sesión de entrenamiento del Explorador

Luis Mario Medrano Zeballos

Carrera de ingeniería en sistemas

Programación II

7 de octubre de 2026

Aplicación de consola en C# con registro, autorización, reglas de entrenamiento, validación de entradas y respuesta ante la anomalía IRIS. Este documento acompaña el proyecto de Visual Studio y presenta su diseño y verificación.

Explorador y realidad asignada

El explorador de referencia es Luis Mario Medrano Zeballos. La realidad propuesta, **LUMEN-07**, es un archivo de ecos temporales: cada exploración descubre rastros de otras sesiones, mientras la interferencia puede comprometer el enlace. El programa permite introducir un nombre de realidad distinto.

Para una demostración normal se pueden introducir 20 años, 80 puntos de energía y 80 de estabilidad. Estos son datos de prueba. La inmersión exige **edad ≥ 16, energía ≥ 40 y estabilidad ≥ 60**. Los umbrales son decisiones ficticias del simulador.

Reglas diseñadas

Operación	Efecto y motivo
Cartografiar un eco	Consume 12 de energía y 8 de estabilidad. Desde la segunda exploración seguida consume 19 y 14: representa sobrecarga y evita repetir sin planificar.
Decodificar IRIS	Consume 18 de energía y 12 de estabilidad; revela información narrativa. Obtener información también implica un riesgo.
Consultar bitácora	No cambia ningún valor ni interrumpe la secuencia de exploraciones. Permite decidir con información actual.
Usar celda	Recupera hasta 22 de energía y resta 4 de estabilidad. Hay 3 celdas; el máximo de energía es 100. No se gasta una celda si ya está al máximo.
Recalibrar ancla	Consume 10 de energía y recupera hasta 18 de estabilidad, sin superar 100. Permite reducir el riesgo a cambio de recursos.
Cerrar enlace	La opción 5 termina voluntariamente. No se permiten acciones después del cierre.

Analizar, recargar o recalibrar con éxito interrumpe la secuencia de exploraciones. Si falta energía, la acción se rechaza sin alterar datos. Esta combinación hace que el jugador elija entre progreso, información y estabilidad.

Decisiones y protocolo de seguridad

Anomalía IRIS y condición crítica

La alerta IRIS se activa cuando **estabilidad ≤ 30 O bien energía ≤ 20 Y estabilidad ≤ 45**. La segunda condición corresponde al nivel élite porque combina dos variables. Además, la penalización por exploraciones seguidas depende del historial de decisiones.

NEXUS corta automáticamente la conexión si **estabilidad ≤ 10 O bien energía = 0 Y estabilidad ≤ 25**. Una alerta no siempre termina la sesión: el jugador puede recalibrar o desconectarse antes de llegar al límite crítico.

El evento **AnomaliaIris** se publica desde la simulación y un manejador de la interfaz muestra la alerta. Solo se emite una vez mientras persiste el mismo episodio de riesgo; puede emitirse de nuevo si el estado se recupera y posteriormente vuelve al riesgo.

Qué se investigó y cómo se aplicó

Se consultó la documentación de Microsoft sobre **int.TryParse**. Este método informa si un texto se convirtió a entero sin usar una excepción para el error de formato. Por eso ABC se rechaza con un mensaje, mientras -500 se rechaza al comprobar el rango, aunque sí sea un entero válido (Microsoft, s. f.-c).

También se revisaron **try, catch y finally**. La lectura cerrada produce EndOfStreamException y termina de forma controlada; un problema de entrada o salida se maneja con IOException. En las pruebas, finally restaura la consola redirigida incluso si una comprobación falla (Microsoft, s. f.-b).

Se investigó el modelo de eventos para separar la detección del riesgo de la presentación de la alerta. En este caso, la simulación detecta y publica; la interfaz recibe el aviso y responde. Así se implementa el desafío opcional sin cambiar el ciclo principal (Microsoft, s. f.-a).

Estructura y dificultad de diseño

Program.cs contiene la bienvenida, el registro y el menú dentro de un **while**. El ciclo continúa mientras Conectada sea verdadero. Simulacion.cs conserva el estado y ejecuta las acciones con **switch**. Pruebas.cs verifica resultados concretos sin depender de paquetes externos.

Las variables de texto representan nombre y realidad; los enteros representan edad, energía, estabilidad y contadores; los booleanos representan autorización y conexión. Los operadores aritméticos actualizan recursos y los operadores lógicos combinan condiciones. Los atributos con modificación privada evitan cambios externos arbitrarios.

Dificultad principal y solución

La dificultad de diseño fue coordinar la recuperación y el peligro sin permitir estados imposibles. Una recarga ilimitada eliminaría las decisiones del jugador; una penalización excesiva impediría recuperarse. La solución fue limitar las celdas a tres, ofrecer recalibración y separar la alerta del corte de emergencia.

Para evitar inconsistencias se verifica primero si una acción puede ejecutarse y luego se actualizan sus efectos. **Las operaciones rechazadas no consumen recursos**. Math.Min y Math.Max mantienen los valores entre 0 y 100. Se usan reglas deterministas, de modo que cada recorrido de prueba sea reproducible.

Cómo abrir y demostrar el proyecto

Extraer el ZIP, abrir NEXUS_Entrenamiento.sln en Visual Studio con soporte de .NET 8 y ejecutar con Ctrl+F5. Registrar los datos, utilizar varias opciones y cerrar con 5. El archivo LEEME.md incluye las instrucciones y todos los umbrales.

Desde una terminal en la carpeta de la solución, las órdenes son:

```
dotnet run --project NexusTraining
dotnet run --project NexusTraining -- --pruebas
```

Resultados de verificación

Se compiló el código con Roslyn del SDK de .NET 8 y se ejecutó una batería de 12 pruebas. La salida completa se adjunta en RESULTADOS_PRUEBAS.txt. Las seis pruebas obligatorias se comprobaron mediante los siguientes escenarios:

Prueba	Datos y resultado observado
1 Inmersión autorizada	Edad 20, energía 80 y estabilidad 80: conexión activa. También se verificó el límite 16/40/60.
2 Inmersión rechazada	Edad 15, energía 39 o estabilidad 59, probados por separado: ingreso rechazado.
3 Operaciones del menú	Desde 80/80: explorar produce 68/72; analizar, 50/60; consultar no modifica; recargar, 72/56; recalibrar, 62/74.
4 Entrada incorrecta	ABC y -500 se rechazan antes de aceptar energía 80; el menú rechaza 99 y acepta 5. No hay cierre inesperado.
5 Condición crítica	Desde 100/60, analizar 3 veces produce 46/24 y alerta IRIS. Dos análisis más producen 10/0 y corte automático.
6 Desconexión	La opción 5 cierra la sesión. Una acción posterior no cambia el estado.

Las comprobaciones adicionales cubrieron límites de 100, reservas agotadas, energía insuficiente, regla élite, fin de entrada, una sesión completa y rechazo de un estado inválido. La edad usada en las pruebas es ficticia.

Resultado de la ejecución

RESULTADO: 12 aprobadas; 0 fallidas.

Las pruebas se ejecutaron por terminal en este entorno. La solución se entrega para abrirla en Visual Studio; no se afirma que se haya probado dentro de su interfaz gráfica.

Referencias y apoyo de IA

Microsoft. (s. f.-a). *Events*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/events/

Microsoft. (s. f.-b). *Exception-handling statements*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements

Microsoft. (s. f.-c). *Int32.TryParse method*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse?view=net-8.0

Registro de apoyo

Se utilizó ChatGPT para proponer reglas, elaborar código, preparar casos de prueba y redactar explicaciones. Las fuentes oficiales se consultaron el 7 de octubre de 2026. Las partes resaltadas en amarillo identifican las condiciones clave para comprender la solución.

Consulta de origen, extracto textual de la consigna compartida: “Desarrolla en C# una aplicación de consola que simule una sesión de entrenamiento de un Explorador NEXUS”. Esta transcripción documenta el pedido; no constituye una captura de pantalla.

Este material corresponde a la misión de NEXUS enviada en la conversación. No se recibieron preguntas de una parte Written ni capturas de prompts para incorporarlas. Si también se exige la entrega diagnóstica anterior, esos elementos deben añadirse para completar sus requisitos.